Hardware/Software Design Description

For

CourtSweeper: A Vision-Based Tennis Ball Retriever with Dual-Roller Intake Mechanism

Version 1.0 approved

Group 13

Date 2026-03-23

Revision History

Name	Date	Reason For Changes	Version
Yilin Zhang	2026-03-17	Initial document creation and LaTeX template configuration.	0.1
Yingrui Sun	2026-03-18	Drafted initial hardware specifications, including dual-roller mechanism and power system.	0.2
Shijia Luo	2026-03-18	Drafted software architecture, including machine vision algorithms and communication protocols.	0.3
Yuwei Zhao	2026-03-23	Comprehensive review and optimization. Finalized system design and approved Baseline 1.0.	1.0

Table of Contents

1 INTRODUCTION

1.1 Purpose

This document describes the preliminary design of the CourtSweeper system. Its primary purpose is to provide a comprehensive architectural overview and detailed engineering specifications, serving as the Baseline 1.0 blueprint for the development team. It details the physical mechanical structures, electronic power systems, computer vision algorithms, and the iOS-based user interface to guide the implementation, integration, and testing phases.

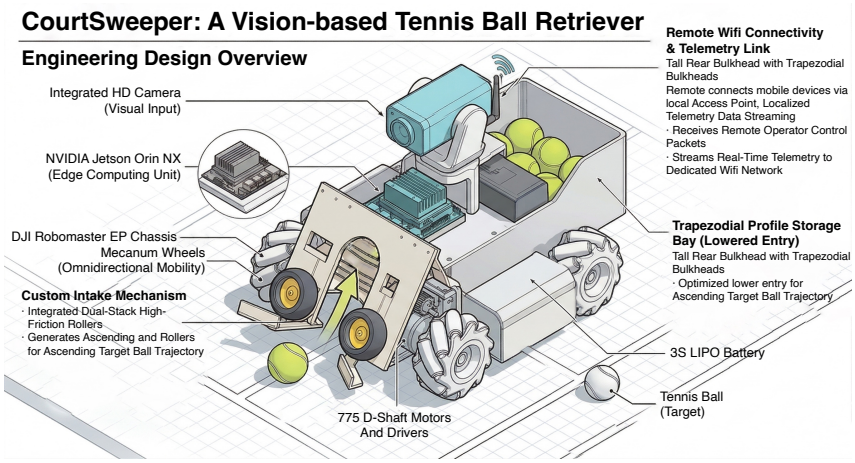

Figure 1: Comprehensive engineering design overview of the CourtSweeper robotic platform.

1.2 Scope

This document describes the design of the CourtSweeper, an intelligent, vision-guided robotic vehicle designed to automate the retrieval of scattered tennis balls in sports facilities. The scope of this system encompasses:

- **Hardware Subsystem:** A hybrid mobile platform utilizing the DJI RoboMaster EP [2] chassis to provide omnidirectional mobility and the primary HD camera feed. It is physically integrated with a custom-built, high-torque dual-roller intake mechanism, which is driven by 775 D-shaft motors and BTS7960 drivers for ball collection, and an NVIDIA Jetson Orin NX [4] acting as the central edge computing unit.
- **Software Subsystem:** An onboard AI pipeline utilizing the RoboMaster-SDK-Ultra [5] (a customized, high-performance fork of the official DJI SDK developed specifically for this project) for vehicle telemetry and camera stream extraction. This is combined with the YOLO [6] object detection model and OpenCV, enabling real-time target recognition, spatial coordinate mapping, and dynamic intake motor control via PWM.

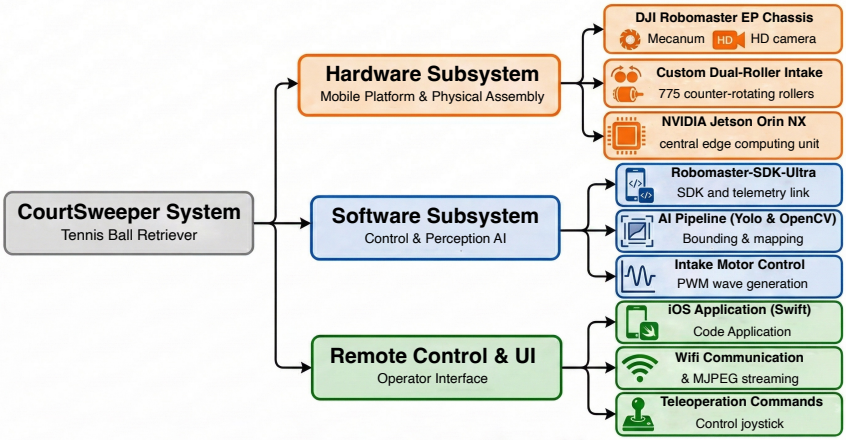

Figure 2: Hierarchical scope tree diagram detailing the overarching architecture of the CourtSweeper system.

- **Remote Control & UI:** A native iOS application [1] developed in Swift, establishing a localized Wi-Fi connection to stream AI-annotated video feeds (via MJPEG) and transmit low-latency teleoperation commands to the Jetson unit.

1.3 Definitions, Acronyms and Abbreviations

Table 1: List of abbreviations and terminology used in the CourtSweeper.

Term	Definition
HSDD	Hardware/Software Design Description.
CS	CourtSweeper, the designated project name for the tennis ball collecting robot.
UI	User Interface, specifically referring to the iOS application in this project.
YOLO	You Only Look Once (specifically yolo26), a SOTA real-time object detection system utilized for high-speed tennis ball recognition.
RoboMaster-SDK-Ultra	A customized, advanced fork of the official DJI SDK (developed by RamessesN), utilized to interface with the RoboMaster EP chassis, execute complex telemetry, and extract raw HD camera feeds.
OpenCV	Open Source Computer Vision Library, utilized for image processing, color space conversion, and bounding box coordinate extraction.
FSM	Finite State Machine, the logic architecture governing the autonomous and manual behavioral states of the robot.
IPC	Inter-Process Communication, a data exchange mechanism between OS processes. The custom SDK circumvents IPC to minimize video latency.
Mecanum Wheels	Omnidirectional wheels on the RoboMaster EP chassis that allow the robot to move in any direction without changing its orientation.
PWM	Pulse Width Modulation, utilized for custom motor speed regulation in the 775 motor-driven intake mechanism.
LiPo	Lithium Polymer, the high-discharge-rate battery chemistry utilized for the heavy-duty 3S actuation power source.

Table 1 continued

Term	Definition
MJPEG	Motion JPEG, a video compression format used to stream AI-processed (bounding-box annotated) video frames to the iOS client with low latency.
HUD	Heads-Up Display, the primary transparent-style layout utilized in the iOS application for concurrent video and telemetry monitoring.
AP Mode	Access Point Mode, allowing the Jetson Orin NX to act as a localized Wi-Fi router for direct mobile connection in outdoor environments.

1.4 References

[1] Apple Inc. *Apple Developer Documentation - Network Framework*. 2026. URL: https://developer.apple.com/documentation.

[2] DJI. *RoboMaster EP Programming Guide and SDK Documentation*. 2020. URL: https://www.dji.com/cn/robomaster-ep.

[3] *Dual Roller Mechanism*. 2024. URL: https://www.bilibili.com/video/BV1bgveeoEiX?vd_source=6621406a814999260edb3d6efe8942da.

[4] Nvidia Inc. *NVIDIA Jetson Orin NX*. 2022. URL: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/.

[5] RamessesN. *Robomaster SDK Ultra*. 2025. URL: https://github.com/RamessesN/Robomaster-SDK-Ultra.

[6] *Ultralytics YOLO26*. 2026. URL: https://docs.ultralytics.com/models/yolo26/.

1.5 Organization of Document

To provide a structured and logical flow of the system architecture, this document is organized into the following major sections:

Table 2: Organization of this Hardware/Software Design Description.

Section	Core Content	Description
Section 2 *Hardware Design*	• DJI RoboMaster EP Chassis • Custom Intake Mechanism • Jetson Orin NX Setup • Power Management System	Provides the physical and electrical blueprint of the robot. It details the integration of the hybrid mobile platform, the custom 775 motor-driven dual-roller mechanism, and the multi-voltage power distribution topology.

Table 2 continued

Section	Core Content	Description
Section 3 *Autonomous Navigation & System Architecture*	• ROS2-Based SLAM Mapping • Nav2 Autonomous Navigation • Hierarchical FSM Design • Class Architecture	Describes the overarching system architecture and autonomous navigation pipeline. It details the ROS2 middleware layer bridging DJI hardware with SLAM and Nav2, the hierarchical two-layer FSM governing mission execution, and the modular class design.
Section 4 *Perception & Reactive Control*	• YOLO Computer Vision Pipeline • Intake Motor Control • Chassis PID Navigation • Post-Ingestion Sector Sweep	Provides the in-depth implementation of the perception and reactive control layer. It covers dataset collection, model training, and real-time YOLO inference; the dual-roller motor firmware and ingestion trigger FSM; the dual-PID chassis controller with blind-zone relay; and the post-ingestion 360-degree sector clearing logic.
Section 5 *UI Design*	• Main Dashboard & Video HUD • Teleoperation & Control Panel • Telemetry & Log Console	Outlines the Swift-based iOS application architecture, detailing the visual presentation layer, network socket communication, and real-time system monitoring protocols.
Section 6 *Theoretical Manual*	• Perception Models & Sensor Theory • Mecanum Kinematics & PID Control • SLAM, AMCL & Path Planning • FSM, Coordinate Transforms, & Protocols	Presents the mathematical foundations and algorithmic principles underpinning each subsystem, including sensor models, control laws, navigation algorithms, communication specifications, and motor drive theory.

2 HARDWARE DESIGN

The hardware architecture of the CourtSweeper system is a hybrid design, integrating a commercial robotic platform with custom-engineered mechanical and computational subsystems. This section details the physical embodiment, sensor configurations, and electrical topologies required to achieve autonomous tennis ball retrieval.

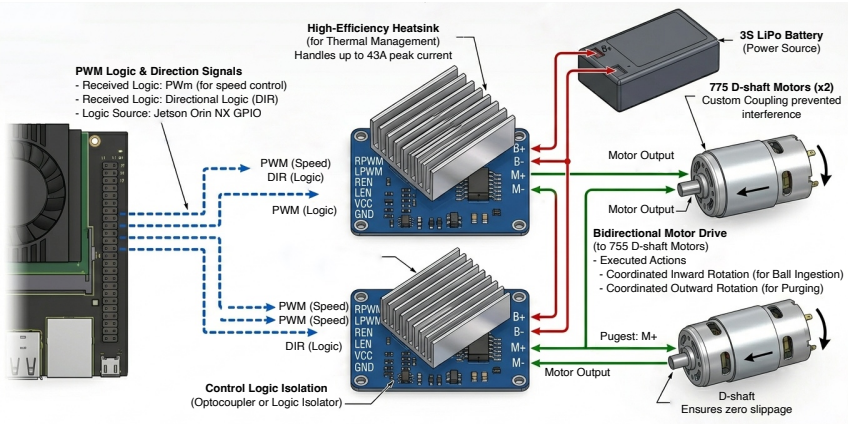

Figure 3: An engineering diagram detailing the BTS7960 high-current drive architecture for the CourtSweeper system, illustrating the connection of an NVIDIA Jetson Orin NX to two BTS7960 motor drivers powered by a 3S LiPo battery to control two 775 D-shaft motors, featuring logic signal connections, power distribution, bidirectional motor drive, and technical callouts for key components like heatsinks and motor features.

2.1 Camera Setup

The primary visual input via the integrated HD camera on the Robomaster EP chassis, which feeds raw video streams to the edge computing unit for target detection. The core hardware parameters of the integrated camera are detailed in Table 3.

Table 3: DJI RoboMaster EP Integrated Camera Specifications

Hardware Parameter	**Specification**
Sensor Type	CMOS 1/4 , Effective Pixels: 5 MP
Field of View (FOV)	120°
Max Video Resolution	FHD: 1080p@30fps / HD: 720p@30fps
Max Video Bitrate	16 Mbps
Max Still Photo Resolution	2560 × 1440

Figure 4: A physical picture of Robomaster EP Camera Module.

2.2 DJI RoboMaster EP Chassis

Figure 5: A physical picture of Robomaster EP Chassis Module.

The foundation of the CourtSweeper is the DJI RoboMaster EP chassis. Selected for its industrial-grade stability and payload capacity, the chassis is equipped with four Mecanum wheels, granting the robot true omnidirectional mobility. This holonomic drive system is crucial for the precise spatial adjustments required to align the front-facing intake mechanism with scattered tennis balls without needing a turning radius.

2.3 Custom Intake Mechanism

To achieve reliable and rapid ball collection, a custom dual-roller intake mechanism [3] was designed and mounted at the front of the chassis. The actuation is driven by two high-torque 775 DC motors (D-shaft configuration, rated at 12V and 4600 RPM).

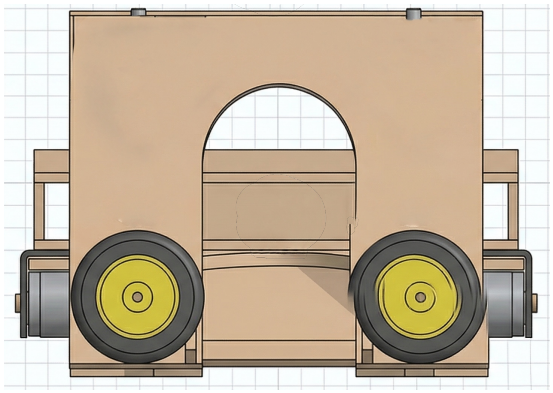

Figure 6: A detailed front-on view of the Courtsweeper dual-roller intake mechanism.

- **Mechanical Coupling:** The motors are mated to 60mm solid rubber wheels featuring high-friction hexagonal grooves via 30mm extended brass couplings. This extended design provides essential clearance to prevent mechanical interference between the rotating wheel hubs and the motor casing, while the D-shaft ensures zero slippage under high torque.
- **Motor Drivers:** Two BTS7960 high-current motor driver modules (capable of handling up to 43A peak current) are utilized to independently control the two motors. They receive Pulse Width Modulation (PWM) and directional logic signals from the central computing unit to execute coordinated inward rotation (for ball ingestion) or outward rotation (for purging).

2.4 LiDAR Sensor Setup

To ensure precise operational boundary gridding and efficient path planning (as outlined in the software architecture Fig.11), the Courtsweeper incorporates a dedicated 2D LiDAR sensor. This sensor provides robust 2D scanning of the court environment and potential obstacles (see §6.1 for the operating principle).

- **Sensor Details:** The selected hardware is the YDLIDAR X3-YB-1 triangulation-based LiDAR. It is mounted at the rear of the chassis and interfaces with the Jetson Orin NX via a customized USB-serial connection.
- **Specifications:** The X3-YB-1 offers a maximum distance range of up to 10 meters, enabling comprehensive court coverage. Key operational parameters include a sampling

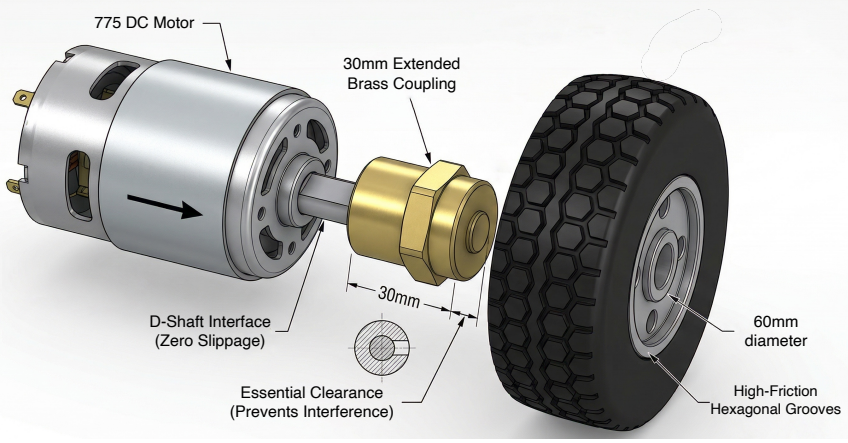

Figure 7: Close-up view of the mechanical coupling. The 30mm extended brass coupling provides critical operational clearance between the 775 motor casing and the high-friction intake roller.

frequency of 4000 Hz, a standard rotating frequency of 7 Hz, and an angular resolution of 0.6°.

Figure 8: A physical picture of the YDLIDAR X3-YB-1 LiDAR sensor.

2.5 Jetson Orin NX Setup

The cognitive core of the system is the NVIDIA Jetson Orin NX. Acting as the Edge Computing Unit, it is responsible for handling computationally intensive tasks locally, thereby eliminating cloud-dependency and minimizing latency. The Jetson module runs the Ubuntu operating system and is configured to:

1. Execute the YOLO object detection pipeline concurrently with OpenCV operations to process the H.264 video stream captured by the DJI chassis.

2. Run the customized RoboMaster-SDK-Ultra to translate bounding box coordinates into proportional navigation commands.
3. Generate precise PWM signals via its GPIO pins to command the BTS7960 drivers.
4. Operate in Access Point (AP) mode to establish a localized Wi-Fi network for the iOS UI client.

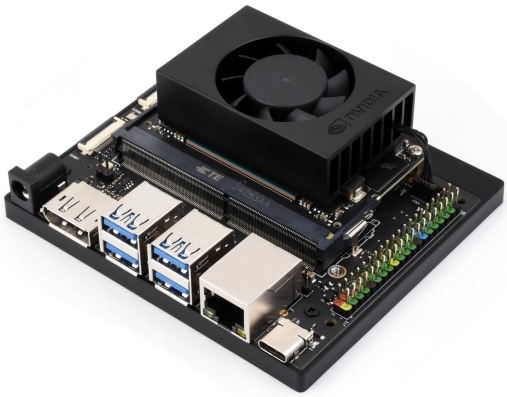

Figure 9: A physical picture of NVIDIA jetson Orin NX.

2.6 Power Management System

Given the diverse voltage requirements and the risk of electromagnetic interference from high-current motors, the CourtSweeper employs an isolated, multi-rail power distribution topology:

- **Mobility Power:** The DJI RoboMaster EP chassis and its onboard camera are powered autonomously by the proprietary DJI Intelligent Battery (3S LiPo, Nominal 10.8V, 2400mAh, 25.92Wh).
- **Actuation Power:** The 775 intake motors and BTS7960 drivers are powered by a dedicated 3S (11.1V) Lithium Polymer (LiPo) battery rated at 5200mAh and 40C discharge. This ensures the motors receive sufficient burst current during ball compression without inducing voltage sags in the logic circuits. A low-voltage buzzer (BX100) is attached to the balance lead for real-time cell monitoring.
- **Logic Power:** The Jetson Orin NX requires a stable 19V input. To prevent logic resets caused by motor back-EMF, it is powered by a galvanically isolated power source.
- **Common Grounding:** Crucially, the ground (GND) planes of the Jetson Orin NX and the BTS7960 motor drivers are tied together to ensure reliable logic-level signal transmission.

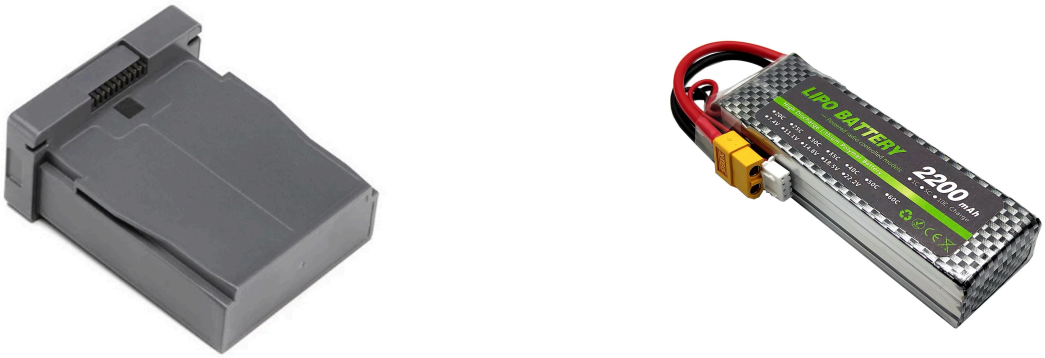

Figure 10: A physical picture of Robomaster EP battery & Motor-driven 3S LiPo battery.

3 SOFTWARE DESIGN: AUTONOMOUS NAVIGATION AND SYSTEM ARCHITECTURE

3.1 Architecture Overview

The software subsystem of CourtSweeper is structured as a hierarchical two-layer Finite State Machine (FSM) architecture, as illustrated in Figure 11. All modules run on the NVIDIA Jetson Orin NX edge computing unit, bridging heterogeneous sensory inputs (DJI camera, RPLidar, IR proximity sensor) with mechanical outputs (Mecanum chassis, dual-roller intake) through a ROS2-based middleware layer.

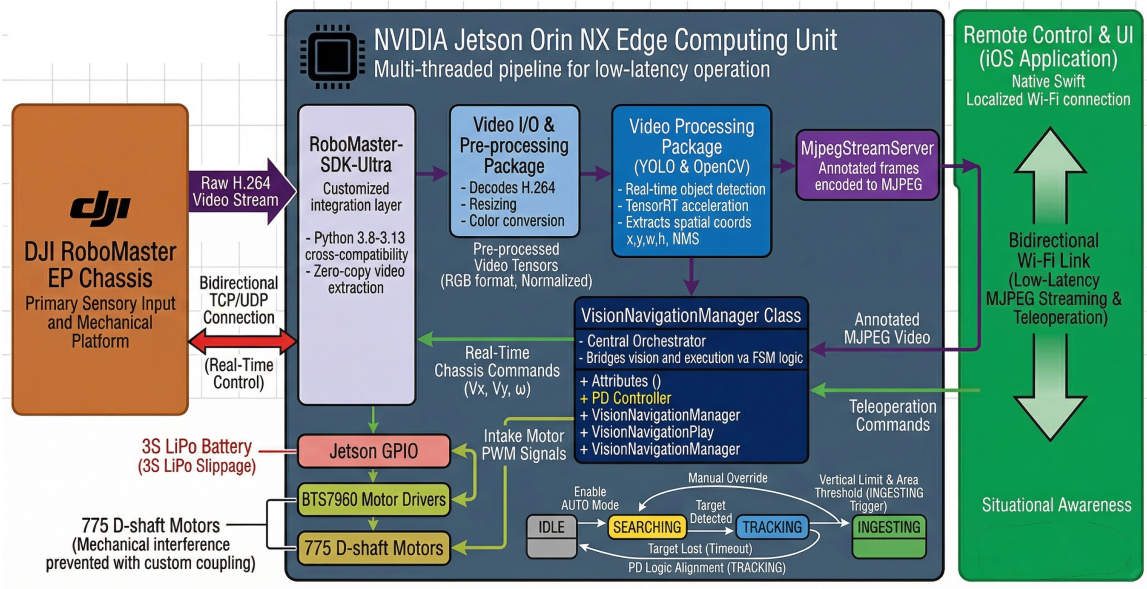

Figure 11: The hierarchical software architecture of CourtSweeper. The upper layer handles iOS interrupt commands, while the lower layer executes the autonomous workflow through three sequential phases.

Every ROS2 bridge node in the system follows a uniform initialization pattern, establishing a 50 Hz control loop timer and subscribing to the `/cmd_vel` topic for navigation command forwarding:

```
class RoboMasterBridge(Node):
    def __init__(self):
        super().__init__('robomaster_bridge')
        self.odom_pub   = self.create_publisher(Odometry, '/odom', 10)
        self.camera_pub = self.create_publisher(CompressedImage,
                            '/camera/image/compressed', 10)
        self.cmd_sub    = self.create_subscription(Twist,
```

```
                                '/cmd_vel', self.cmd_vel_callback, 10)
        self.tf_broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.02, self.timer_callback)
```

The critical ROS2 data streams flowing through this middleware layer are summarized in Table 4.

Table 4: Core ROS2 topics bridging the DJI chassis and the navigation/vision stacks.

Topic	**Message Type**	**Rate**	**Direction**
`/odom`	`nav_msgs/Odometry`	50 Hz	Chassis → ROS2
`/camera/image/compressed`	`sensor_msgs/CompressedImage`	10 Hz	Chassis → ROS2
`/camera/camera_info`	`sensor_msgs/CameraInfo`	10 Hz	Chassis → ROS2
`/cmd_vel`	`geometry_msgs/Twist`	On demand	ROS2 → Chassis
`/scan`	`sensor_msgs/LaserScan`	~10 Hz	LiDAR → ROS2
`/tf` (odom→base_footprint)	`tf2_msgs/TFMessage`	50 Hz	Bridge → ROS2

3.2 Chassis Navigation & DJI SDK Integration

To interface with the mobile platform, the system implements **RoboMaster-SDK-Ultra**, a high-performance community fork of the official DJI Python SDK. The robot connects to the Jetson via Wi-Fi in Station (STA) mode and is set to free-motion mode before any chassis commands are issued:

```
from robomaster_ultra import robot

ep_robot = robot.Robot()
ep_robot.initialize(conn_type="sta")
ep_robot.set_robot_mode(mode="free")
ep_chassis = ep_robot.chassis
```

Velocity commands arriving via `/cmd_vel` are converted to per-wheel RPM targets using the Mecanum differential mixing equations (see §6.2 for the kinematic derivation). A scaling factor of 100 maps the normalized Twist velocities to the chassis wheel-speed range:

```
def cmd_vel_callback(self, message):
    vx = message.linear.x;  vy = message.linear.y
    vw = message.angular.z

    left_speed  = vx - vy - vw
    right_speed = vx + vy + vw
    factor = 100

    ep_chassis.drive_wheels(
        w1=int(right_speed * factor),  # right-front
        w2=int(left_speed  * factor),  # left-front
```

```
        w3=int(left_speed  * factor),  # left-rear
        w4=int(right_speed * factor))  # right-rear
```

A 300 ms communication watchdog is enforced: if no `/cmd_vel` message arrives within this window while the robot is moving, all wheel outputs are zeroed to prevent runaway behavior.

3.3 SLAM-Based Mapping Pipeline

The first phase of autonomous operation constructs a 2D occupancy grid map of the tennis court (theoretical principles in §6.4). The mapping pipeline integrates an RPLidar A1M8, the `slam_toolbox` ROS2 package, and a custom bridge node.

3.3.1 RoboMaster Chassis Bridge (map_generation.py)

The `RoboMasterBridge` ROS2 node publishes chassis odometry and camera data into the ROS2 ecosystem. Chassis position (x, y) and yaw θ are subscribed from the DJI SDK, converted from Euler angles to quaternions, and published at 50 Hz as TF transforms and `/odom` messages:

```
def timer_callback(self):
    yaw_rad = -math.radians(c_sub.latest_yaw)
    yaw_rad = math.atan2(math.sin(yaw_rad), math.cos(yaw_rad))
    qx, qy, qz, qw = euler_to_quaternion(yaw_rad)

    t = TransformStamped()
    t.header.frame_id = 'odom'
    t.child_frame_id = 'base_footprint'
    t.transform.translation.x = -c_sub.latest_x
    t.transform.translation.y =  c_sub.latest_y
    t.transform.rotation.x = qx; t.transform.rotation.y = qy
    t.transform.rotation.z = qz; t.transform.rotation.w = qw
    self.tf_broadcaster.sendTransform(t)

    odom = Odometry()
    odom.header.frame_id = 'odom'
    odom.child_frame_id  = 'base_footprint'
    odom.pose.pose.position.x = -c_sub.latest_x
    odom.pose.pose.position.y =  c_sub.latest_y
    self.odom_pub.publish(odom)
```

The camera stream is captured at 10 Hz, JPEG-compressed with quality factor 60, and published alongside a pinhole `CameraInfo` message. A 300 ms watchdog in the `/cmd_vel` subscriber ensures safety during navigation.

3.3.2 LiDAR Integration & Filtering

The RPLidar A1M8 2D laser scanner is launched via `rplidar_ros`:

```
ros2 launch rplidar_ros rplidar_a1_launch.py \\
  serial_port:=/dev/ttyCH341USB0
```

A `laser_filters` scan-to-scan filter chain is applied to suppress noise and constrain the effective measurement range. Static TF transforms (`base_footprint` → `base_link` → `laser`) are published to align the LiDAR frame with the robot body frame.

3.3.3 SLAM Solver (slam_toolbox)

The `slam_toolbox` package in online asynchronous mode fuses the filtered laser scans with chassis odometry to incrementally build a 2D occupancy grid. The operator manually teleoperates the robot around the court perimeter while the solver performs scan-to-map matching and loop closure:

```
ros2 launch slam_toolbox online_async_launch.py \\
  slam_params_file:=~/ros2_ws/config/mapper_params_online_async.yaml
```

Upon completion, the map is persisted to disk for subsequent reuse during autonomous navigation:

```
ros2 run nav2_map_server map_saver_cli -f custom_court
```

3.4 Nav2-Based Autonomous Navigation

Once the occupancy grid map is saved, the system transitions to autonomous navigation using the **Nav2** (Navigation2) framework (algorithmic details in §6.4). Nav2 provides AMCL-based localization, a global planner, a local controller, and behavior-tree-driven mission execution.

3.4.1 Localization and Planning

Nav2's Adaptive Monte Carlo Localization (AMCL) module continuously estimates the robot's pose (x, y, θ) on the pre-built map using LiDAR scan and odometry inputs. The planner server is configured with the Smac Hybrid-A* global planner and the Regulated Pure Pursuit (RPP) local controller, running at their default Nav2 parameters. The localization and navigation stacks are launched with the saved map:

```
# Localization
ros2 launch nav2_bringup localization_launch.py \
  map:=/home/nvidia/ros2_ws/maps/custom_court.yaml \
  use_sim_time:=False

# Navigation
ros2 launch nav2_bringup navigation_launch.py \
  map:=/home/nvidia/ros2_ws/maps/custom_court.yaml \
  use_sim_time:=False
```

3.4.2 RoboMaster Navigation Bridge (map_navigation.py)

During the navigation phase, a second ROS2 node, `RoboMasterNavigation`, replaces the mapping bridge. Its `timer_callback` is nearly identical to the mapping bridge, with one critical difference: the yaw angle and position coordinates are not negated, preserving the raw chassis coordinate orientation to match the saved map's convention:

```
# Navigation bridge: coordinate sign differs from mapping bridge
yaw_rad = math.radians(c_sub.latest_yaw)     # no negation
corrected_x = c_sub.latest_x                  # no negation
corrected_y = c_sub.latest_y
```

Nav2's planner server publishes `/cmd_vel` directly; the bridge intercepts these messages and forwards them to the DJI chassis through the same Mecanum mixing pipeline with a 300 ms watchdog.

3.4.3 Waypoint-Based Coverage Strategy

The operator defines a sequence of navigation waypoints covering the traversable playing area in an S-shaped (boustrophedon) pattern. Nav2's behavior tree executes each waypoint sequentially. During waypoint traversal, the YOLO vision pipeline runs concurrently: if a tennis ball is detected, navigation is interrupted and the reactive retrieval pipeline (Section 4) takes over.

3.5 Video I/O & Pre-processing

The raw H.264 video feed from the DJI camera is decoded by the RoboMaster-SDK-Ultra's `libmedia_codec_ultra` backend. Each frame is resized to 640×360 pixels at 10 Hz and JPEG-compressed (quality 60) before being published to `/camera/image/compressed`. The pre-processing pipeline and full YOLO detection module are detailed in Section 4.1.

3.6 Video Processing (YOLO & OpenCV)

Real-time tennis ball detection uses the YOLO architecture with key operational parameters listed in Table 5. The system auto-detects the optimal inference backend at startup using a priority-ordered hardware probe:

```
DEVICE = (
    "cuda" if torch.cuda.is_available() else
    "mps"  if torch.mps.is_available()  else
    "xpu"  if torch.xpu.is_available()  else
    "cpu"
)
```

The detection output drives the chassis alignment controller via the horizontal pixel offset `visual_error_x`. The full dataset collection workflow, model training hyperparameters, and TensorRT deployment pipeline are covered in Section 4.1.

Table 5: YOLO detection pipeline parameters.

Parameter	**Value**
Base model	YOLOv6-Nano (`yolo26n.pt`)
Inference resolution	640×360
Confidence threshold	0.25
Deployment backends	TensorRT FP16 (Jetson), PyTorch (macOS)
Nearest-target strategy	$\arg\max(y_2)$ (largest bottom y-coordinate)
Control output	`visual_error_x` = target_x $-$ $\frac{\text{width}}{2}$
Frame-boundary filter	10-pixel margin from edge (relinquish lock)

3.7 Hierarchical Finite State Machine Design

The robot's behavioral logic is governed by a two-layer hierarchical Finite State Machine (formal definition in §6.5), illustrated in Figure 12. The upper layer handles iOS-originated interrupt commands with unconditional priority; the lower layer executes the autonomous mission workflow.

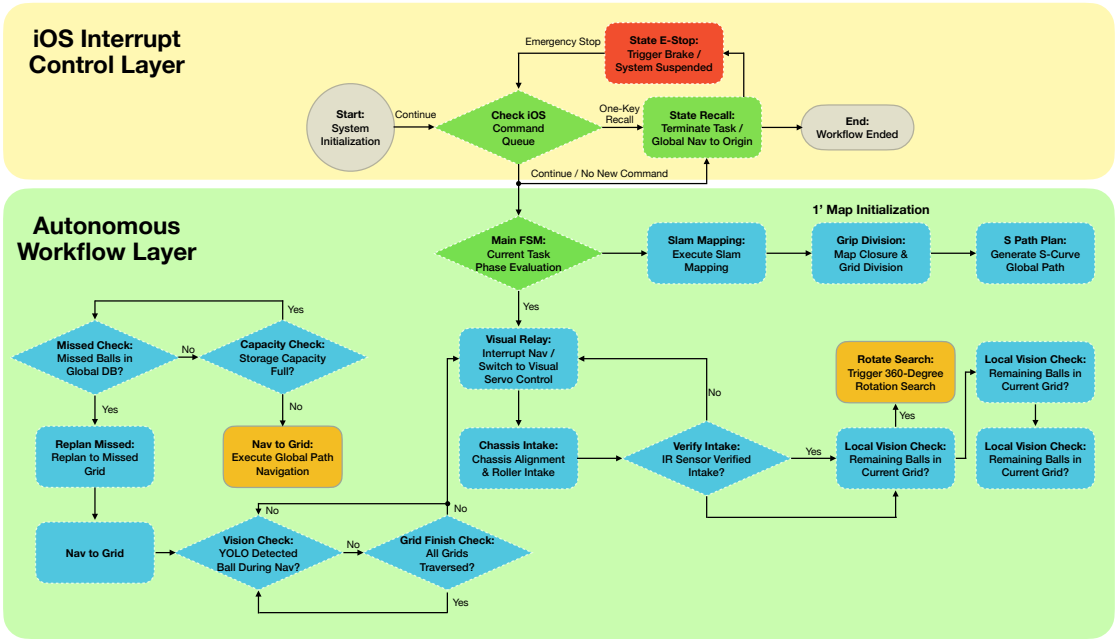

Figure 12: Hierarchical FSM flow diagram. The upper interrupt control layer preempts the lower autonomous workflow at any time. The autonomous layer progresses through three phases: mapping, waypoint navigation, and visual closed-loop retrieval with 360° post-ingestion sector sweep.

3.7.1 iOS Interrupt Control Layer (Upper FSM)

The upper FSM continuously polls the iOS command queue over UDP. Two high-priority interrupt commands preempt any ongoing autonomous behavior:

1. **Emergency Stop (E-STOP):** Immediately engages chassis braking and suspends all autonomous logic until the operator issues a resume command.

2. **One-Touch Recall:** Terminates the current mission and invokes the Nav2 planner to navigate the robot to the predefined origin pose.

3.7.2 Autonomous Workflow Layer (Lower FSM)

The lower FSM transitions through three sequential mission phases:

Phase 1 — Mapping & Global Planning. SLAM toolbox constructs the occupancy grid while the operator teleoperates the robot around the court perimeter. Once the map is saved, an S-shaped waypoint sequence is loaded into the Nav2 waypoint follower.

Phase 2 — Waypoint Navigation & Interrupt Handling. Before dispatching each waypoint, two pre-navigation checks execute: a missed-ball database lookup for previously detected but unrecovered ball locations, and a storage capacity check that triggers recall if the onboard bay is full. If clear, Nav2 navigates to the next waypoint. During traversal, the YOLO pipeline runs concurrently; upon detection, navigation is interrupted and Phase 3 activates. When all waypoints are visited, the system triggers a recall.

Phase 3 — Visual Closed-Loop Retrieval. This phase is implemented in `chassis_ctrl.py` as an implicit state machine driven by three Boolean flags defined at module level:

```
visual_error_x = 0
target_locked  = False    # whether a ball is currently locked
scan_needed    = False    # whether to perform a 360-degree scan
no_ball_counter = 0       # consecutive frames without detection
```

The main control loop transitions between behavioral modes based on these flags and the real-time IR distance sensor reading (millimetres). The transition logic at the top of the loop is:

```
target_valid = vn.target_locked
need_scan    = vn.scan_needed
dist = ds.latest_distance if ds.latest_distance is not None else 8848

if target_valid:
    is_chasing_sequence = True
    last_valid_dist = dist
elif need_scan:
    is_chasing_sequence = False
    last_valid_dist = 8848
```

When `is_chasing_sequence` is active, the controller executes three priority-ordered behaviors based on the current sensor reading. Key numerical thresholds are listed in Table 6.

After confirmed ingestion, the robot executes a 360° in-place rotational scan. If YOLO detects another ball during the scan, the chasing sequence re-engages (looping back to visual servoing). If the scan completes with no detections, the sector is marked as cleared, the global ball-coordinate database is updated, and control returns to the main FSM for the next waypoint dispatch. The detailed implementation of the reactive control pipeline is presented in Section 4.

Table 6: FSM Phase 3 operational parameters implemented in `chassis_ctrl.py` and `video_node.py`.

Stage	**Parameter**	**Value**
Target acquisition	Consecutive no-ball frames to trigger scan	15
Visual servoing	Turn PID (`pid_turn`)	$K_p = 0.15, K_i = 0.03, K_d = 0.03$
	Turn PID output limits	$[-50, 50]$
	Forward PID (`pid_forward`)	$K_p = 0.5, K_i = 0.1, K_d = 0.05$
	Forward PID output limits	$[-40, 40]$
	Alignment-priority threshold	$\lvert\text{error\_x}\rvert > 40$ px
	Open-loop cruise speed	30 (forward)
Blind-zone relay	Target-lost distance guard	dist < 450 mm
	Last-valid proximity guard	last_valid_dist < 300 mm
Ingestion trigger	Pre-spin distance threshold	≤ 200 mm
	Motor pre-spin duty	90 (∼35%)
	Approach speed during ingestion	30 (forward)
Ingestion confirm	Disappearance distance threshold	> 250 mm
	Consecutive disappearance frames	10 (200 ms)
	Hard timeout	3.0 s
360° sector sweep	In-place rotation speed	30 (turn)

3.8 Class Design

CourtNavigationManager Class Description

The central orchestrator of the autonomous workflow is the `CourtNavigationManager` class, which bridges the ROS2 navigation stack, the computer vision pipeline, and the hardware actuation interfaces through the hierarchical FSM. At the implementation level, the Phase 3 reactive FSM logic resides in `chassis_ctrl.py` using the Boolean flag mechanism described above; `CourtNavigationManager` coordinates the higher-level phase transitions and Nav2 dispatch.

Attribute	**Description**
Class Name	`CourtNavigationManager`
Aggregated Classes	`YoloDetector`, `Nav2PlannerClient`, `DjiChassisController`, `MotorPwmDriver`, `IrSensorReader`
Associated Classes	`MjpegStreamServer`, `IosCommandReceiver` (UDP listener for iOS interrupts)
Data Members	• `master_state`: Enum (`MAPPING`, `NAVIGATING`, `INTERCEPTING`, `RETRIEVING`, `RECALLING`, `E_STOP`). • `robot_pose_global`: Pose (x, y, θ) obtained from Nav2 AMCL localization. • `waypoint_queue`: Ordered S-shaped sequence of navigation goals. • `missed_ball_db`: List of (x, y) coordinates of detected but unrecovered balls. • `storage_capacity`: Current ball count in the onboard bay.
Member Functions	• `process_ios_interrupt()`: Receives UDP commands from the iOS app (E-STOP / Recall / Continue). • `update_master_state()`: High-level FSM transitions based on vision events, waypoint progress, and capacity thresholds. • `generate_coverage_path()`: Generates S-shaped waypoint sequence from the SLAM map. • `dispatch_nav2_goal()`: Sends a target pose to Nav2 and monitors execution. • `handle_visual_intercept()`: Suspends Nav2, switches to PD visual servoing, triggers intake on proximity confirmation. • `perform_360_scan()`: In-place rotation with YOLO monitoring; loops back to retrieval if a new ball is found.
Processing	Operates in a dedicated high-priority thread. Fuses Nav2 pose with YOLO detections, executes the hierarchical FSM, dispatches velocity commands, and manages motor triggering.

4 SOFTWARE DESIGN: PERCEPTION AND REACTIVE CONTROL

4.1 Computer Vision & Object Detection Module

This module is the core perception layer of the CourtSweeper system, responsible for real-time video frame capture, tennis ball target detection, and providing pixel-space coordinate offset to the downstream motion control module. The system is built on the YOLO (You Only Look Once) deep learning object detection framework, supports multi-platform deployment (macOS, Linux/Jetson), and is optimized with TensorRT acceleration for embedded edge computing scenarios. Theoretical foundations — including the camera pinhole model, IBVS error, YOLO architecture and loss functions, and LiDAR triangulation — are presented in §6.1.

4.1.1 Module Architecture

The overall workflow of the computer vision module can be summarized into four stages: dataset collection and annotation, model training, cross-platform model deployment, and real-time object detection. The system receives the real-time video stream from the RoboMaster gimbal camera, performs YOLO inference to output the bounding box coordinates of tennis ball targets, and then computes the horizontal offset of the target relative to the image center (visual error), which serves as the input signal for the chassis and gimbal control loops.

The module consists of the following core files:

`cv_config.py`	Device-adaptive and path configuration
`dataset_collect.py`	Dataset collection tool
`model_training.py`	YOLO model training script
`pt2onnx.py`	PyTorch model export to ONNX format
`onnx2engine.sh`	ONNX model conversion to TensorRT engine
`video_node.py`	Real-time object detection main node

4.1.2 Dataset Collection

The quality of the dataset directly affects the detection accuracy of the model. We developed the `dataset_collect.py` tool to capture tennis ball images in actual court environments using the built-in RoboMaster camera.

The collection program communicates with the robot via a direct Wi-Fi connection (AP mode), starts a 720P HD video stream, and displays the resized frames at 1280×960 pixels in real time. A crosshair is drawn at the center of the frame to assist composition, helping the collector position the tennis ball in the central region. Pressing the `s` key triggers a snapshot save, followed by a brief white flash as visual feedback; pressing `q` exits the program. Images are saved to the `./dataset/tennis_v2/` directory with a timestamp-plus-sequence naming scheme (e.g., `tennis_20250519_143025_0001.jpg`).

```
1 def img_collect():
```

```
    save_dir = "./dataset/tennis_v2"
    os.makedirs(save_dir, exist_ok=True)

    ep_robot = robot.Robot()
    ep_robot.initialize(conn_type="ap")
    ep_robot.set_robot_mode(mode="free")
    ep_camera = ep_robot.camera
    ep_camera.start_video_stream(display=False,
                                 resolution=camera.STREAM_720P)

    count = 0
    while True:
        img = ep_camera.read_cv2_image()
        if img is not None:
            img_resized = cv2.resize(img, (1280, 960))
            cv2.line(display_img, (center_w - 20, center_h),
                     (center_w + 20, center_h), (0, 255, 0), 2)
            cv2.line(display_img, (center_w, center_h - 20),
                     (center_w, center_h + 20), (0, 255, 0), 2)
            if key == ord('s'):
                cv2.imwrite(filename, img_resized)

```

After collection, annotation tools such as LabelImg are used to label the tennis balls in the images with bounding boxes, generating YOLO-format label files (`.txt`). A `data.yaml` dataset configuration file is then written, specifying the training and validation set paths and class names.

4.1.3 Model Training

The object detection model is trained using the Ultralytics YOLO framework. Starting from the YOLOv6-Nano (`yolo26n.pt`) pre-trained weights as a foundation, transfer learning is applied to rapidly adapt the model from general-purpose detection capability to the single tennis ball class.

The training hyperparameters are configured as follows:

Initial learning rate (lr0)	0.01
final learning rate (lrf)	0.01
Optimizer	auto (SGD with momentum=0.937)
Weight decay	0.0005
Warmup epochs	3.0
Data augmentation	Mosaic stitching, random horizontal flip (fliplr=0.5), HSV color perturbation, random affine transformation (scale=0.5, translate=0.1), RandAugment automatic augmentation, random erasing (erasing=0.4)
Early stopping patience	100 epochs

```
from ultralytics import YOLO
```

```

def train_model():
    model = YOLO("./weight/yolo26n.pt")   # load pre-trained model

    results = model.train(
        data=DATA_YAML,                   # dataset configuration file
        epochs=200,                       # training epochs
        imgsz=640,                        # input image size
        batch=16,                         # batch size (adjust by GPU memory)
        name=MODEL_NAME,                  # model name
        device=DEVICE,                    # runtime device
        workers=2,                        # data loading threads
        project="./runs/train_tennis",    # output directory
        exist_ok=False                    # do not overwrite existing experiment
    )
```

Device selection is handled by the adaptive logic in `cv_config.py`, which probes hardware accelerators in priority order: CUDA (NVIDIA GPU), MPS (Apple Silicon), XPU (Intel GPU), and falls back to CPU if none is available:

```
DEVICE: str = (
    "cuda" if torch.cuda.is_available() else
    "mps" if torch.mps.is_available() else
    "xpu" if torch.xpu.is_available() else
    "cpu"
)
```

The artifacts produced during training include: the best-weight file `best.pt`, the last-epoch weight `last.pt`, F1 curve plot, PR curve plot, confusion matrix, training batch visualization samples, and a `results.csv` file recording per-epoch losses and metrics for subsequent analysis and tuning.

4.1.4 Cross-Platform Model Deployment

To accommodate the inference performance requirements of different deployment environments, the system implements a model conversion pipeline from PyTorch to ONNX to TensorRT.

PyTorch → ONNX Conversion: Using the built-in export functionality of Ultralytics, the `.pt` weights can be converted to the open ONNX intermediate representation format with a single line of code:

```
from ultralytics import YOLO

model = YOLO("./pt/tennis.pt")
model.export(format="onnx")
```

The generated `.onnx` file can be used for inference on any platform supporting ONNX Runtime, ensuring the model's universality and portability.

ONNX → TensorRT Engine: On edge computing platforms such as the NVIDIA Jetson, to fully leverage GPU inference acceleration, NVIDIA TensorRT's `trtexec` tool is used to compile the ONNX model into a highly optimized TensorRT engine (`.engine` file), with FP16 half-precision inference enabled to substantially boost the inference frame rate with negligible loss in detection accuracy:

```
/usr/src/tensorrt/bin/trtexec           \
--onnx=onnx/tennis_v2.onnx              \
--saveEngine=engine/tennis_v2.engine    \
--fp16
```

At inference time, the appropriate model format is automatically selected based on the runtime platform:

```
# Mac Config
model = YOLO("src/vision/mlmodel/pt/tennis_v2.pt")

# Linux Config (Jetson + TensorRT)
# model = YOLO("src/vision/mlmodel/engine/tennis_v2.engine")
# model.names = {0: 'tennis ball'}
```

4.1.5 Real-Time Object Detection Pipeline

`video_node.py` is the runtime core of the entire vision module. It is responsible for acquiring video frames from the robot camera, invoking the YOLO model for object detection, computing the target's offset from the frame center, and overlaying rich debugging information on the frame.

Video Stream Acquisition.

The video stream is started at 360P resolution via the RoboMaster SDK to balance image quality and frame rate. Each received frame is resized to 640×360 pixels before being fed into the YOLO model for processing.

```
def video_capture(ep_camera):
    global latest_frame, annotated_frame, running

    ep_camera.start_video_stream(display=False,
                                 resolution=camera.STREAM_360P)

    while running:
        img = ep_camera.read_cv2_image()
        if img is not None:
            img = cv2.resize(img, (640, 360))
            latest_frame = img
            annotated_frame = yolo_predict(img)
            cv2.imshow("on Live", annotated_frame)
```

Object Detection and Filtering.

YOLO inference uses a confidence threshold of 0.25 to filter out low-quality detections. When multiple detection results are present in the frame (e.g., multiple tennis balls), the system compares the bottom Y-coordinate (y2 value) of each bounding box to select the ball closest to the camera — a larger y2 value indicates that the target is lower in the frame, i.e., nearer to the camera. This strategy ensures that the robot prioritizes locking onto and tracking the nearest target ball.

Once the target is selected, the horizontal coordinate of its bounding box center, `target_x`, is subtracted from the frame centerline `center_x` to compute the horizontal visual deviation `visual_error_x`. This offset is the key input quantity for subsequent chassis rotation control:

$$\text{visual\_error\_x} = \text{target\_x} - \frac{\text{width}}{2}$$

```python
def yolo_predict(frame):
    results = model(frame, conf=0.25, device=cfg.DEVICE,
                    verbose=False)
    annotated = results[0].plot()

    height, width = frame.shape[:2]
    center_x = width // 2

    if len(results[0].boxes) > 0:
        boxes = results[0].boxes

        closest_box = None
        max_y2 = -1
        for box in boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            if y2 > max_y2:
                max_y2 = y2
                closest_box = box  # select the nearest target

        x1, y1, x2, y2 = map(int, closest_box.xyxy[0])
        target_x = (x1 + x2) // 2
        target_y = (y1 + y2) // 2
        visual_error_x = target_x - center_x
```

Target Locking and Scanning Mechanism.

The system maintains three states: `Locked` (target locked), `Searching` (normal search in progress), and `Scanning` (360-degree scan for ball search). When no tennis ball is detected for 15 consecutive frames (`no_ball_counter >= 15`), the system automatically triggers the `scan_needed` flag, notifying the chassis control module to perform an in-place rotation scan to re-locate the ball. Upon detecting a target, the counter resets to zero and the state returns to locked. Additionally, if the target approaches the edge of the frame (within 10 pixels of the boundary), the system treats it as about to leave the field of view and actively relinquishes the lock to avoid ineffective tracking.

```
if not (10 <= target_x <= width - 10) or \
   not (10 <= target_y <= height - 10):
    target_x = None
    target_y = None
    visual_error_x = 0
    target_locked = False

# no ball detected
no_ball_counter += 1
if no_ball_counter >= 15:  # 15 consecutive frames without target → trigger scan
↪  mode
    scan_needed = True
```

4.1.6 Visual Overlay & Debug Information

To facilitate real-time debugging and status monitoring, the system overlays the following information on each frame in OSD (On-Screen Display) format:

- **Resolution** (RES): Current frame dimensions, displayed in red
- **Frame Rate** (FPS): Sliding-average frame rate over the last 10 frames, displayed in green
- **Distance**: Real-time ranging value from the distance sensor module (unit: cm), displayed in blue
- **Status**: Current tracking state — `Locked` (green) / `Searching` or `Scanning` (orange)
- **Center Crosshair**: A gray crosshair marking the geometric center of the frame
- **Target Marker**: When a target is locked, a solid blue circle is drawn at the target center with the horizontal offset value annotated beside it
- **Bounding Box**: YOLO detection boxes are rendered as yellow rectangles

```
# Resolution
cv2.putText(annotated, f"RES: {width}x{height}",
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
            1, (0, 0, 255), 2, lineType=cv2.LINE_AA)
# Frame Rate
cv2.putText(annotated, f"FPS: {int(avg_fps)}",
            (10, 60), cv2.FONT_HERSHEY_SIMPLEX,
            1, (0, 255, 0), 2, lineType=cv2.LINE_AA)
# Distance
cv2.putText(annotated, f"Distance: {distance_text}",
            (10, 90), cv2.FONT_HERSHEY_SIMPLEX,
            1, (255, 0, 0), 2, lineType=cv2.LINE_AA)
# Status
state_text = "Locked" if target_locked else \
             ("Scanning" if scan_needed else "Searching")
```

```
cv2.putText(annotated, f"Status: {state_text}",
            (10, 120), cv2.FONT_HERSHEY_SIMPLEX,
            1, color, 2, lineType=cv2.LINE_AA)
```

4.1.7 Inter-Module Data Exchange

The computer vision module shares data with other system modules through module-level global variables:

- `visual_error_x`: The horizontal pixel offset of the target relative to the frame center, used by the chassis control module (`chassis_ctrl.py`) to compute the rotational angular velocity
- `target_locked`: The target lock status flag, used by the main control logic to determine the current tracking phase
- `scan_needed`: The 360-degree scan ball-search request flag, triggering the chassis to execute a scanning rotation maneuver
- `latest_frame / annotated_frame`: The raw and annotated frames, available for other modules that require image data

Simultaneously, the vision module also reads the `latest_distance` variable provided by the distance sensor module, displaying real-time tennis ball distance information on the OSD and thereby achieving fused multi-sensor information presentation.

4.1.8 Summary

The CourtSweeper computer vision module utilizes the YOLO deep learning framework to achieve a complete closed-loop pipeline encompassing data collection, model training, cross-platform deployment, and real-time inference. Through the nearest-target filtering strategy, target locking and scanning search mechanism, and horizontal visual deviation computation, the module provides reliable perceptual input for the robot's autonomous tracking and ball retrieval behaviors. The model deployment pipeline supports the three-stage PyTorch→ONNX→TensorRT conversion, enabling efficient inference at FP16 half-precision on edge devices such as the NVIDIA Jetson, thereby satisfying real-time performance requirements.

4.2 Intake Motor Control Module

The intake motor control module is the critical actuation unit of the CourtSweeper system. It is responsible for driving the dual-roller mechanical structure to ingest the ball into the onboard storage bay once the vision system has locked onto a tennis ball target. This module spans both low-level hardware driving and high-level decision logic, forming a complete control chain from signal generation to intelligent triggering. The underlying PWM theory, BTS7960 H-bridge topology, and watchdog mechanism are formalised in §6.8.

4.2.1 Hardware Driver Architecture

The intake mechanism is driven by two 775 DC motors (D-shaft configuration, rated at 12 V and 4600 RPM), each coupled to a 60 mm high-friction solid rubber roller via a 30 mm extended brass coupling. Motor driving is handled by two BTS7960 high-current full-bridge driver modules (capable of handling up to 43 A peak current), independently controlling the direction and speed of the left and right rollers.

The BTS7960 driver modules interface with the lower-level controller via a three-wire interface (enable pin EN, forward PWM pin RPWM, and reverse PWM pin LPWM). The system utilizes a dedicated 3S (11.1 V) 5200 mAh Lithium Polymer (LiPo) battery (discharge rate 40C) to power the driver modules, ensuring sufficient burst current for the motors during ball compression while preventing voltage sag interference to the main control logic circuits. Low-level PWM signal generation is handled by an Arduino microcontroller, which receives commands from the Jetson Orin NX via USB serial.

4.2.2 Low-Level Firmware Design

The low-level Arduino firmware (`roller_setting.ino`) is responsible for converting serial commands into PWM signals, enabling real-time control of the dual BTS7960 drivers. The firmware defines the control interfaces for the left and right motors through a pin-mapping scheme, as detailed in Table 7.

Table 7: 775 Motor Driver Pin Mapping Table

Motor	Enable (EN)	Forward PWM (RPWM)	Reverse PWM (LPWM)
Left Motor	Pin 4	Pin 5	Pin 6
Right Motor	Pin 7	Pin 9	Pin 10

The firmware main loop operates in blocking serial-read mode: when data is available in the serial buffer, it calls `Serial.parseInt()` to continuously parse the left and right motor speed values (formatted as `"L_speed,R_speed\n"`), then invokes the `driveMotor()` function to convert the values into corresponding PWM signals. This function clamps the speed to the range $[-255, 255]$ and determines the rotation direction based on the sign: positive values activate the RPWM channel, negative values activate the LPWM channel, and a zero value pulls the enable pin low to cut off the output.

```
void driveMotor(int enPin, int fwdPin, int revPin, int speed) {
  speed = constrain(speed, -255, 255);

  if (speed == 0) {
    digitalWrite(enPin, LOW);
    analogWrite(fwdPin, 0);
    analogWrite(revPin, 0);
  } else if (speed > 0) {
    digitalWrite(enPin, HIGH);
    analogWrite(fwdPin, speed);
```

```
    analogWrite(revPin, 0);
  } else {
    digitalWrite(enPin, HIGH);
    analogWrite(fwdPin, 0);
    analogWrite(revPin, abs(speed));
  }
}
```

To ensure system safety, the firmware incorporates a communication timeout protection mechanism. Upon each successful command parsing, a timestamp is recorded. If no new command is received within 500 ms while the motors are in motion, `stopMotor()` is automatically invoked to zero all motor outputs, preventing the motors from running idle during a communication loss.

```
void loop() {
  if (Serial.available() > 0) {
    int speedL = Serial.parseInt();
    int speedR = Serial.parseInt();

    if (Serial.read() == '\n') {
      driveMotor(L_EN, L_RPWM, L_LPWM, -speedL);
      driveMotor(R_EN, R_RPWM, R_LPWM, speedR);

      lastRecvTime = millis();

      if (speedL != 0 || speedR != 0) {
        isMoving = true;
      }
    }
  }

  if (isMoving && (millis() - lastRecvTime > 500)) {
    stopMotor();
    isMoving = false;
  }
}
```

4.2.3 High-Level Control Interface

On the Jetson Orin NX side, the Python module `roller_drive.py` encapsulates serial communication with the Arduino. Its core is the `RollerMotor` class: the constructor establishes a connection via the PySerial library at 115200 baud and waits 2 s for initialization, while the `set_speed()` method formats the left and right wheel speeds as a CSV string and writes it to the serial port.

```
class RollerMotor:
    def __init__(self, port='', baudrate=115200):
        logger.info(f"Connecting to motor on {port} ...")
```

```
        try:
            self.ser = serial.Serial(port, baudrate, timeout=1)
            time.sleep(2)
            logger.info("Connection successful!")
        except Exception as e:
            logger.error(f"Failed to connect to serial port. Error details: {e}")
            exit(1)

    def set_speed(self, left_speed, right_speed):
        command = f"{int(left_speed)},{int(right_speed)}\n"
        self.ser.write(command.encode('utf-8'))
        logger.info(f"Roller Speed: L = {left_speed}, R = {right_speed}")

    def close(self):
        self.set_speed(0, 0)
        time.sleep(0.1)
        self.ser.close()
        logger.info("Connection closed.")
```

This design treats the Python side as a pure command-dispatching layer, offloading all PWM timing generation and direction logic to the Arduino firmware, allowing the upper layer to focus on decision scheduling without concern for time-sensitive low-level signal generation. The `close()` method ensures safe shutdown by issuing a zero-speed command before closing the serial port.

4.2.4 FSM Integration & Intake Trigger Logic

The motor control module is integrated into the chassis control main loop (`chassis_ctrl.py`) and is triggered by the Finite State Machine (FSM) based on visual perception results. The triggering and execution of the intake logic can be decomposed into the following three phases:

Phase 1: Target Distance Pre-Assessment

When the vision system locks onto a tennis ball target (`target_locked = True`), the system enters the chasing sequence (`is_chasing_sequence`). The PD controller drives the chassis to continuously align with and approach the target, while simultaneously reading the front-facing distance sensor value in real time. When the target distance falls within 200 mm, the system determines that the ball has entered the reachable range of the rollers and immediately transitions to the pre-spin phase.

Phase 2: Motor Pre-Spin & Slow Advance

Upon entering the pre-spin phase, the motors begin rotating at a medium speed (both set to 90, approximately 35% duty cycle), while the chassis continues a slow forward advance at speed 30. This design ensures that the rollers already possess rotational kinetic energy at the moment of contact with the ball, preventing jamming or bouncing caused by acceleration from a standstill.

```
if dist <= 200:
    logger.info(f"     ({dist/10:.1f}cm)      ...")
```

```

    if intake_motor:
        intake_motor.set_speed(90, 90)

    drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)
```

Phase 3: Ingestion Confirmation & Timeout Protection

After the motors start, the system enters an ingestion confirmation window of at most 3 s. The front-facing distance sensor value is continuously monitored at a 20 ms sampling period: if the ball is successfully ingested, the distance sensor reading will jump beyond the measurement range (> 250 mm). The system performs 10 consecutive disappearance confirmations (totaling 200 ms) to prevent false positives. Once the ball is confirmed ingested, it immediately stops the chassis and motors and resets the chasing state, returning the FSM to the searching state to resume the coverage path.

If the distance sensor reading does not exhibit any jump within the 3 s window, the system determines that a jam or missed-ball anomaly has occurred and likewise stops execution while outputting a warning log, preventing motor overheating from prolonged stalling.

```
swallow_start_time = time.time()
missing_count = 0

while time.time() - swallow_start_time < 3.0:
    current_dist = ds.latest_distance if ds.latest_distance is not None else 8848

    if current_dist > 250:
        missing_count += 1
    else:
        missing_count = 0

    if missing_count >= 10:
        is_swallowed = True
        break

    time.sleep(0.02)

if is_swallowed:
    logger.info("     ")
else:
    logger.info("       ")

chassis_stop(ep_chassis)

if intake_motor:
    intake_motor.set_speed(0, 0)
```

Blind-Zone Relay Mechanism

Additionally, the system implements a blind-zone relay logic: when the ball distance has already been reduced to within 300 mm but the visual target is briefly lost (with the distance sensor still

reading within the valid 450 mm range), the chassis maintains straight forward advance without immediately aborting the chasing sequence. This mechanism effectively compensates for the visual blind zone caused by the camera's limited field of view at close range, improving the success rate of near-field ingestion.

Post-Ingestion Sector Sweep and FSM Re-Entry

Following a confirmed ball ingestion, the system does not immediately resume waypoint navigation. Instead, it executes a 360° in-place rotational scan at a fixed angular velocity (30 RPM) to check whether additional tennis balls remain within the current court sector. During the rotation, the YOLO pipeline runs continuously: if a new ball is detected, the chasing sequence re-engages, pulling execution back to the visual servoing phase. This looping mechanism ensures that all balls within a local sector are retrieved before the robot moves on, avoiding expensive back-and-forth navigation.

If the rotation completes with no further detections, the system determines that the current sector has been fully cleared. The sector status is updated in the global ball-coordinate database, and control returns to the main FSM. The FSM then dispatches the next waypoint from the Nav2 coverage path (or triggers a recall if all waypoints have been visited), as described in the hierarchical FSM design in Section 3.7.

4.3 Proximity-Based Chassis Navigation Module

This module combines real-time proximity sensing with a closed-loop PID chassis controller, responsible for executing the reactive retrieval stage of ball capture: alignment, approach, and physical ingestion. It bridges the visual perception pipeline (Section 4.1) with the intake motor actuator (Section 4.2) through the phase-3 reactive control logic of the hierarchical FSM (Section 3.7), simultaneously fusing two independent data streams — the image-plane centroid offset from the YOLO detector, and millimeter-resolution distance readings from the RoboMaster EP's onboard infrared sensor. The Mecanum kinematics, dual-PID control laws, and blind-zone relay logic are derived in the Theoretical Manual (§6.2–§6.3).

4.3.1 Distance Sensing Submodule

Hardware Interface via RoboMaster-SDK-Ultra.

Distance measurement relies on the RoboMaster EP's built-in infrared proximity sensor, accessed through **RoboMaster-SDK-Ultra**. This SDK is a community-maintained high-performance fork of the official DJI Python SDK, extending compatibility to Python 3.7 through 3.13 and bundling `libmedia_codec_ultra` for zero-copy H.264 decoding. Installation on the Jetson Orin NX proceeds as follows:

```
# Automated installation (recommended)
git clone https://github.com/RamessesN/Robomaster-SDK-Ultra
cd Robomaster-SDK-Ultra
sudo chmod 755 ./installer.sh && ./installer.sh

# Or manual installation on Ubuntu/Debian
```

```
sudo apt update && sudo apt install ffmpeg libopus-dev
pip install -e .
cd robomaster_lib/libmedia_codec_ultra && pip install -e .
cd ../../pybind11 && pip install -e .
```

After installation, the sensor API remains consistent with the official DJI documentation, but the import must come from the patched namespace:

```
from robomaster_ultra import robot   # replaces: from robomaster import robot
```

Subscription Design.

Distance data is obtained through the SDK's asynchronous subscription callback mechanism. A dedicated background thread invokes `sub_distance(freq=10, callback=...)` to register a 10 Hz polling loop; upon each callback trigger, the raw sensor reading (unit: millimetres) is written to a module-level shared variable:

```
# src/distance/distance_sub.py

latest_distance: float | None = None

def get_distance(ep_sensor):
    """Register the 10 Hz distance subscription."""
    ep_sensor.sub_distance(freq=10, callback=sub_data_handler_distance)

def sub_data_handler_distance(sub_info):
    """Callback: atomically update the shared distance register."""
    global latest_distance
    latest_distance = sub_info[0]   # unit: millimetres
```

The sentinel value `None` is used to distinguish "sensor not yet initialized" from a legitimate zero reading. In all chassis controller logic, a `None` reading is substituted with the symbolic constant `8848` (the height of Mount Everest in metres, employed here as an equivalent infinitely-far distance value), ensuring the controller defaults to a pure vision-guided mode in the absence of sensor data.

4.3.2 PID Controller Configuration

Two independent PID controllers respectively govern the two degrees of freedom required during the approach process:

```
from simple_pid import PID

# Lateral alignment: drives image-plane centroid error to zero
pid_turn = PID(0.15, 0.03, 0.03, setpoint=0)
pid_turn.output_limits = (-50, 50)      # RPM-equivalent units
```

```

# Longitudinal approach: drives sensor distance to 10 mm (contact)
pid_forward = PID(0.5, 0.1, 0.05, setpoint=10)
pid_forward.output_limits = (-40, 40)
```

- **`pid_turn`** receives the pixel-space lateral offset `visual_error_x` produced by the YOLO centroid extractor (a positive value indicates the target is to the right of the frame center). Its output is negated before application, such that a target on the right produces a clockwise chassis rotation.
- **`pid_forward`** receives the infrared distance reading (millimetres) and drives the robot toward a target distance of 10 mm — i.e., physical contact with the ball. When the distance is below 800 mm, this controller overrides the open-loop cruise speed (30 RPM); above 800 mm, the chassis advances at a fixed cruise speed to reduce approach time.

4.3.3 Chasing Sequence State Tracking

The Boolean flag `is_chasing_sequence` decouples the visual locking phase from the physical approach phase. Once the YOLO pipeline sets `target_locked` to `True`, this flag is latched high, and the last valid distance reading is saved to `last_valid_dist`. The flag is cleared only when the vision module issues `scan_needed = True` — indicating complete target loss following a successful ingestion or timeout.

This design prevents the distance sensor from falsely triggering the intake motors due to irrelevant obstacles briefly entering the sensor field of view before a visual lock has been established.

```
if target_valid:
    is_chasing_sequence = True
    last_valid_dist = dist
elif need_scan:
    is_chasing_sequence = False
    last_valid_dist = 8848
```

4.3.4 Multi-Stage Approach & Ingestion Logic

When `is_chasing_sequence` is active, the controller executes three priority-ordered behaviors based on the current sensor reading.

Phase 1 — Cruise Tracking (`dist` > 800 mm).

Both PID loops operate simultaneously. The turn controller corrects lateral deviation; when the centroid error exceeds 40 pixels, the forward speed is limited to 5 RPM to ensure rotational alignment stabilizes before resuming advance:

```
turn_speed = -pid_turn(error_x)
if abs(error_x) > 40:
    forward_speed = 5           # prioritise alignment over advance
```

```
else:
    if dist < 800:
        forward_speed = -pid_forward(dist)   # PID-controlled approach
    else:
        forward_speed = 30      # open-loop cruise
drive_chassis(ep_chassis, forward_speed, turn_speed)
```

Phase 2 — Blind-Zone Relay (target lost and `dist` < 450 mm).

The camera's minimum focusing distance together with the physical offset between the camera and the intake rollers collectively form a "blind zone" — the ball disappears from the video frame before entering the rollers. The relay logic bridges this blind zone: if the target was last confirmed within 300 mm (`last_valid_dist < 300`) and the distance sensor still reads below 450 mm, the chassis maintains straight forward advance at 30 RPM without visual feedback:

```
elif not target_valid and dist < 450 and last_valid_dist < 300:
    logger.info(
        f"Blind-zone relay "
        f"(current: {dist/10:.1f} cm, "
        f"last valid: {last_valid_dist/10:.1f} cm) "
        "-- maintaining straight advance..."
    )
    drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)
```

Phase 3 — Ingestion Trigger (`dist` ≤ 200 mm).

When the sensor confirms the ball has entered the 200 mm range, the intake motors pre-spin at full speed and the chassis advances at a fixed 30 RPM. A sliding-window detector then continuously monitors the ball disappearance signal: if the distance reading exceeds 250 mm for 10 consecutive 20 ms cycles (`missing_count ≥ 10`), the ball is determined to have been successfully ingested. A 3-second hard timeout guards against jamming or missing:

```
if dist <= 200:
    logger.info(
        f"Target entered pre-ingestion zone "
        f"({dist/10:.1f} cm) -- pre-spinning motors..."
    )
    if intake_motor:
        intake_motor.set_speed(90, 90)
    drive_chassis(ep_chassis, forward_speed=30, turn_speed=0)

    swallow_start_time = time.time()
    missing_count = 0
    is_swallowed = False

    while time.time() - swallow_start_time < 3.0:
        current_dist = ds.latest_distance \
            if ds.latest_distance is not None else 8848
```

```
        if current_dist > 250:
            missing_count += 1
        else:
            missing_count = 0
        if missing_count >= 10:
            is_swallowed = True
            break
        time.sleep(0.02)

    if is_swallowed:
        logger.info("Ball successfully ingested into storage bay.")
    else:
        logger.info("Warning: advance timeout -- possible jam or miss.")

    chassis_stop(ep_chassis)
    if intake_motor:
        intake_motor.set_speed(0, 0)
    is_chasing_sequence = False
    last_valid_dist = 8848
    time.sleep(0.5)
    continue
```

4.3.5 Chassis Drive Primitives

The Mecanum wheel drive equation converts the two scalar commands `forward_speed` and `turn_speed` into per-wheel RPM targets. Since the DJI SDK's `drive_wheels(w1, w2, w3, w4)` interface uses the right-front / left-front / left-rear / right-rear wheel ordering convention, the differential mixing is computed as follows:

```
def drive_chassis(ep_chassis, forward_speed, turn_speed):
    left_speed  = forward_speed + turn_speed
    right_speed = forward_speed - turn_speed
    ep_chassis.drive_wheels(
        w1=right_speed,   # right-front
        w2=left_speed,    # left-front
        w3=left_speed,    # left-rear
        w4=right_speed    # right-rear
    )

def chassis_stop(ep_chassis):
    ep_chassis.drive_wheels(w1=0, w2=0, w3=0, w4=0)
    time.sleep(0.02)     # brief settle pause before next command
```

During pure straight-line motion, the two sides receive equal speeds; when `turn_speed` is non-zero, one side's speed increases while the other decreases, producing an in-place rotation superimposed on the translational velocity. The 20 ms settling delay in `chassis_stop` prevents new burst commands from being issued before the previous wheel-speed packet is confirmed by the chassis firmware.

4.3.6 Idle & Search Behavior

When no tracking task is in progress, the controller falls back to two passive behaviors driven by the vision module:

```
else:
    if need_scan:
        # Rotate in place to sweep the camera FOV
        drive_chassis(ep_chassis, forward_speed=0, turn_speed=30)
    else:
        chassis_stop(ep_chassis)
```

The `need_scan` flag is triggered by the finite state machine's SEARCHING state, indicating that coverage path following has been temporarily suspended, pending local re-detection. The in-place rotation at a fixed 30 RPM sweeps the 120° camera field of view across the surrounding area; once a new target is detected, the chasing sequence automatically re-engages.

5 USER INTERFACE DESIGN

This section describes the user interface design of the CourtSweeper iOS application. Developed natively in Swift using the SwiftUI framework, the mobile application serves as the primary remote command center and implements the **iOS Interrupt Control Layer** of the hierarchical FSM (Section 3.7). It connects directly to the Jetson Orin NX's localized Wi-Fi hotspot, providing real-time situational awareness, manual teleoperation capabilities, high-priority interrupt commands, and system telemetry monitoring.

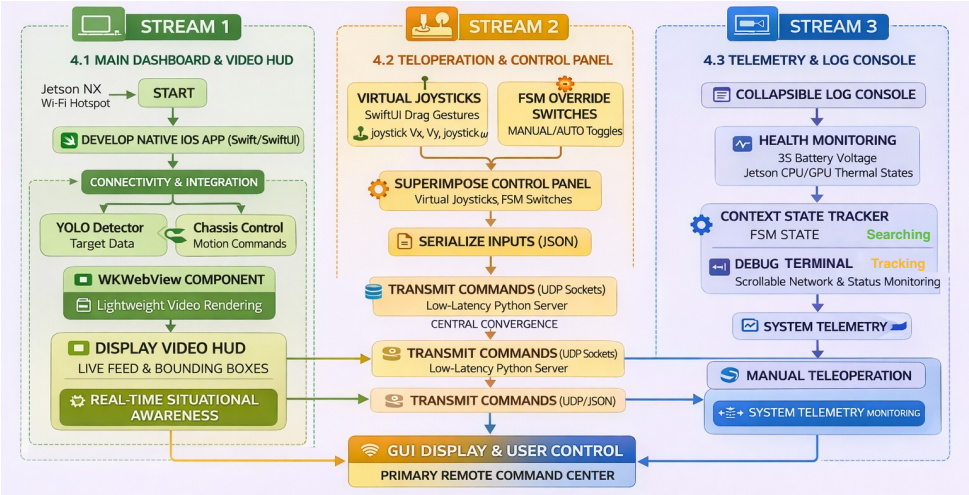

Figure 13: The software architecture and data flow for the CourtSweeper iOS application, illustrating the centralized SwiftUI core connected via bidirectional Wi-Fi and UDP sockets to external Jetson computing units, and featuring functional breakdowns of main dashboard, teleoperation, and telemetry modules.

5.1 Main Dashboard & Video HUD

The primary interface is designed as a landscape-oriented Heads-Up Display (HUD) to maximize the operator's field of view. The core component of this view is the video presentation layer.

- **Video Streaming Rendering:** Instead of relying on complex and latency-prone H.264 decoders on the client side, the app utilizes a lightweight `WKWebView` component to render the MJPEG video stream broadcasted by the Jetson unit.
- **AR Overlay:** The AI-processed bounding boxes, target crosshairs, and ball coordinates are overlaid directly onto the video feed, providing the operator with intuitive, real-time feedback on the YOLO perception module's performance.

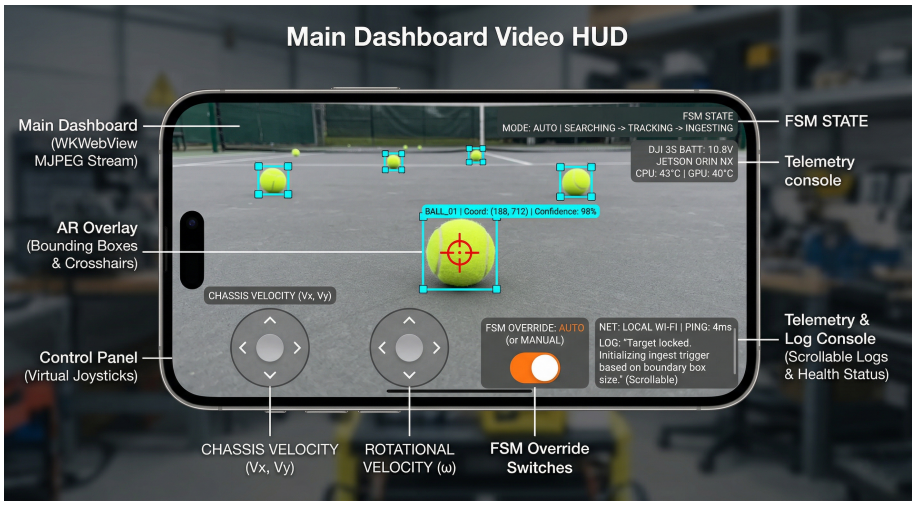

Figure 14: User interface layout of the CourtSweeper iOS application. The primary HUD renders the low-latency MJPEG video stream overlaid with real-time YOLO bounding boxes. The lower panel provides virtual joysticks for holonomic teleoperation and manual override switches for the system's Finite State Machine (FSM).

5.2 Teleoperation & Control Panel

Superimposed on the lower sections of the HUD is the Control Panel, designed for ergonomic thumb access on an iPhone or iPad.

- **Virtual Joysticks:** Implemented via SwiftUI drag gestures, the left joystick dictates the omnidirectional translational movement (V_x, V_y) of the Mecanum chassis, while the right joystick controls rotational velocity (ω).
- **FSM Override Switches:** A prominent set of toggle buttons implements the upper-layer FSM interrupt commands described in Section 3.7. The `E-STOP` button transmits an emergency stop command that immediately engages chassis braking and suspends all autonomous logic. The `RECALL` button triggers a one-touch recall, commanding Nav2 to navigate the robot back to the origin pose. The `AUTO / MANUAL` toggle switches the robot between autonomous mission execution and manual teleoperation mode.
- **Command Transmission:** Control inputs are serialized into lightweight JSON packets and transmitted to the Jetson's Python server via low-latency UDP sockets using Apple's native `Network` framework.

5.3 Telemetry & Log Console

To facilitate debugging and monitor system health during field tests, a collapsible Log Console is integrated into the interface.

- **Health Monitoring:** Displays critical hardware telemetry parsed from the DJI SDK, including the 3S battery voltage (preventing over-discharge) and the Jetson's CPU/GPU thermal states.
- **State Machine Tracker:** A dynamic label continuously reflects the robot's current FSM state from the lower-layer autonomous workflow (e.g., *MAPPING* → *NAVIGATING* → *INTERCEPTING* → *RETRIEVING* → *SCANNING*), as well as any active upper-layer interrupt state (*E-STOP* or *RECALLING*).
- **Debug Terminal:** A scrolling text view (implemented via `ScrollView` in SwiftUI) outputs real-time network connection statuses, ping latency, and critical error logs thrown by the Python backend.

6 THEORETICAL MANUAL

This section presents the mathematical and algorithmic foundations underpinning each subsystem of the CourtSweeper. It formalises the sensor models, kinematic equations, control laws, navigation algorithms, communication protocols, and actuation principles introduced in the preceding design sections (§2–§5).

6.1 Perception Models

6.1.1 Camera Pinhole Model

The system adopts the standard pinhole camera model. Let the camera intrinsic matrix be:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad P = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{1}$$

For the CourtSweeper, the intrinsic parameters are approximated as $f_x = f_y = W$ and $(c_x, c_y) = (W/2, H/2)$, where $W \times H$ is the resized frame dimension (typically 640×360). The lens is assumed to be distortion-free (Brown-Conrady model with all $k_i = 0$).

Table 8: Camera Intrinsic Parameters

Parameter	Value	Min.	Typical	Max.	Units
f_x, f_y (focal length)	W	320	640	1280	px
c_x (principal point x)	$W/2$	160	320	640	px
c_y (principal point y)	$H/2$	90	180	360	px
Distortion	`plumb_bob`	—	$k_i = 0$	—	—

IBVS Visual Error.

The system employs Image-Based Visual Servoing (IBVS): the control error is computed directly in the image plane rather than in 3D Cartesian space. Given the YOLO-detected target centre (x_t, y_t) and the frame centre (c_x, c_y):

$$e_x(t) = x_t - c_x \tag{2}$$

A positive e_x indicates the target lies to the right of the optical axis. This scalar is fed as the process variable to the lateral PID controller (see §6.3).

Nearest-Target Depth Proxy.

When multiple tennis balls appear in a single frame, the system selects the nearest candidate using the bottom Y-coordinate heuristic:

$$\text{target} = \arg\max_{\text{box}_i} \ (y_{2,i}) \tag{3}$$

Larger y_2 values correspond to objects lower in the frame, which under the ground-facing camera assumption implies closer proximity.

6.1.2 YOLO Object Detection Theory

Architecture.

The system deploys YOLOv6-Nano (`yolo26n.pt`), a single-stage anchor-free detector comprising three components:

- **Backbone:** EfficientRep — a re-parameterisation-friendly CNN for multi-scale feature extraction.
- **Neck:** Rep-PAN — a Path Aggregation Network with re-parameterisation blocks.
- **Head:** Decoupled head with independent classification and localisation branches.

Loss Functions.

Training minimises the composite loss:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}}\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{DFL}} \tag{4}$$

where:

- $\mathcal{L}_{\text{CIoU}}$ (Complete IoU loss) regresses bounding boxes by jointly optimising overlap area, centre-point distance, and aspect ratio consistency.
- $\mathcal{L}_{\text{BCE}}$ (Binary Cross-Entropy) penalises class misclassification.
- $\mathcal{L}_{\text{DFL}}$ (Distribution Focal Loss) models box edge positions as discrete probability distributions for anchor-free regression.
- $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, $\lambda_{\text{dfl}} = 1.5$ are empirically weighted.

Training Hyperparameters.

Inference Parameters.

Model Conversion Pipeline.

For edge deployment on the NVIDIA Jetson Orin NX, the model undergoes a three-stage optimisation:

$$\text{PyTorch (.pt)} \xrightarrow{\text{export}} \text{ONNX (.onnx)} \xrightarrow{\texttt{trtexec}} \text{TensorRT (.engine, FP16)} \tag{5}$$

The FP16 half-precision engine reduces inference latency by approximately 2× with negligible detection accuracy degradation on the Jetson's Ampere GPU.

Table 9: YOLO Training Hyperparameters

Hyperparameter	**Value**	**Min.**	**Typical**
Epochs	200	50	200–300
Input size (imgsz)	640	320	640
Batch size	16	4	16–64
Initial LR ($lr0$)	0.01	0.001	0.01
Final LR factor (lrf)	0.01	0.001	0.01
Optimiser	SGD	—	SGD (momentum = 0.937)
Weight decay	0.0005	0	0.0005
Warmup epochs	3.0	0	3.0
Mosaic augmentation	1.0	0	1.0
Horizontal flip probability	0.5	0	0.5
HSV perturbation	(0.015, 0.7, 0.4)	—	—
Random erasing probability	0.4	0	0.4
Early-stopping patience	100	10	50–100
Automatic Mixed Precision (AMP)	ON	—	ON

Table 10: YOLO Inference Parameters

Parameter	**Value**	**Min.**	**Typical**
Confidence threshold	0.25	0.05	0.25–0.50
NMS IoU threshold	0.7	0.3	0.5–0.7
Inference resolution	640×360	—	—
Input channels	3 (RGB)	—	—

6.1.3 LiDAR Triangulation Principle

The YDLIDAR X3-YB-1 operates on the laser triangulation principle. An infrared laser emitter projects a spot onto the target surface; the reflected spot is imaged by a CMOS sensor at a known baseline offset from the emitter. The distance d to the target is recovered by solving the triangle formed by the laser axis, the lens optical axis, and the reflected ray path:

$$d = \frac{b \cdot f}{\Delta x} \tag{6}$$

where b is the baseline distance, f the receiver lens focal length, and Δx the spot displacement on the CMOS array. The sensor head rotates at 7 Hz to achieve 360° scanning coverage.

Table 11: LiDAR Operating Parameters

Parameter	**Min.**	**Typical**	**Max.**	**Units**	**Remarks**
Sampling rate	—	4000	—	Hz	Points per second
Scan frequency	—	7	—	Hz	Motor rotation
Angular resolution	0.6	—	—	°	At 7 Hz
Range	0.05	—	10	m	Indoor, 80% reflectivity
Minimum filtering threshold	—	20	—	mm	`MIN_DIST_MM`

6.1.4 IR Proximity Sensing

The RoboMaster EP chassis provides a built-in infrared proximity sensor. Range data $d_{\mathrm{IR}}(t)$ is polled at 10 Hz via the SDK subscription callback and used as a continuous scalar input (unit: millimetres) for longitudinal PID control and ingestion stage triggering.

Table 12: IR Distance Sensor Parameters

Parameter	Min.	Typical	Max.	Units
Polling frequency	—	10	—	Hz
Valid range (operational)	10	—	8848	mm
Pre-ingestion threshold	—	200	—	mm
Blind-zone range	—	< 450	—	mm
Ingestion-confirmation threshold	—	> 250	—	mm
Sentinel (uninitialised)	—	8848	—	mm

The sentinel value 8848 mm (height of Mount Everest in metres) is used to represent an infinitely far reading when the sensor has not yet been initialised, causing the controller to default to a pure vision-guided mode.

6.2 Mecanum Wheel Kinematics

6.2.1 3-DOF Full Holonomic Inverse Kinematics (Nav2 Mode)

For autonomous navigation with Nav2, the Twist velocity command $\mathbf{v} = (v_x, v_y, \omega_z)$ is decomposed into per-wheel speeds using the standard Mecanum inverse kinematics:

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = \gamma \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & -1 \\ 1 & -1 & -1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \tag{7}$$

where $\gamma = 100$ is an empirical RPM scaling factor. Explicitly:

$$\omega_{\text{right}} = v_x + v_y + \omega_z \tag{8}$$

$$\omega_{\text{left}} = v_x - v_y - \omega_z \tag{9}$$

with the final wheel assignments:

$$\begin{aligned} \text{w1 (right-front)} &= \omega_{\text{right}} \times 100 \\ \text{w2 (left-front)} &= \omega_{\text{left}} \times 100 \\ \text{w3 (left-rear)} &= \omega_{\text{left}} \times 100 \\ \text{w4 (right-rear)} &= \omega_{\text{right}} \times 100 \end{aligned} \tag{10}$$

6.2.2 2-DOF Simplified Inverse Kinematics (Reactive Mode)

For visual servoing, the control is reduced to two scalar commands — forward speed v_f and turn speed v_t:

$$\begin{aligned} \omega_{\text{left}} &= v_f + v_t \\ \omega_{\text{right}} &= v_f - v_t \end{aligned} \tag{11}$$

This formulation sacrifices lateral strafing capability but allows the dual-PID controller to directly govern approach-while-aligning behaviour with two degrees of freedom.

6.2.3 Primitive Motion Analysis

Table 13: Wheel-Speed Decomposition for Fundamental Motions (2-DOF Model)

Motion	ω_{left}	ω_{right}
Pure forward ($v_t = 0$)	v_f	v_f
Pure rotation ($v_f = 0$)	$+v_t$	$-v_t$

Wheel Numbering Convention.

The DJI SDK `drive_wheels(w1, w2, w3, w4)` interface adopts the following mapping:

Table 14: DJI SDK Wheel Convention

w1	**w2**	**w3**	**w4**
Right-Front	Left-Front	Left-Rear	Right-Rear

6.3 PID Control Theory

The reactive chassis controller employs two independent PID regulators in the standard parallel form:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt} \tag{12}$$

where $e(t)$ is the control error at time t, and (K_p, K_i, K_d) are the proportional, integral, and derivative gains respectively.

6.3.1 Lateral Alignment PID (`pid_turn`)

This controller drives the target's image-plane centroid offset e_x (Equation 2) to zero, aligning the robot with the ball.

The output is negated before application: a target on the right ($e_x > 0$) produces a negative turn command, i.e., clockwise rotation.

Table 15: Lateral PID Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
K_p	—	0.15	—	RPM/px	Proportional gain
K_i	—	0.03	—	RPM/(px · s)	Integral gain
K_d	—	0.03	—	RPM · s/px	Derivative gain
Setpoint	—	0	—	px	Centred on optical axis
Output lower	-50	—	—	RPM	Max left-turn speed
Output upper	—	—	+50	RPM	Max right-turn speed

6.3.2 Longitudinal Approach PID (`pid_forward`)

This controller regulates the IR-measured distance $d(t)$ toward a 10 mm setpoint (physical contact with the ball).

Table 16: Longitudinal PID Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
K_p	—	0.5	—	RPM/mm	Proportional gain
K_i	—	0.1	—	RPM/(mm · s)	Integral gain
K_d	—	0.05	—	RPM · s/mm	Derivative gain
Setpoint	—	10	—	mm	Ball contact distance
Output lower	-40	—	—	RPM	Max reverse speed
Output upper	—	—	+40	RPM	Max forward speed

6.3.3 Dual-PID Visual Servoing Architecture

The two controllers operate in a priority-ordered cascade:

1. **Cruise tracking** ($d > 800$ mm): Both PID loops active. If $|e_x| > 40$ px, forward speed is clamped to 5 RPM to prioritise lateral alignment over advance.
2. **PID approach** ($d \leq 800$ mm): The `pid_forward` output overrides the open-loop cruise speed.
3. **Blind-zone relay**: When visual tracking is lost (e_x unavailable) but both $d < 450$ mm and last-valid distance $d_{\text{last}} < 300$ mm, the chassis maintains a straight 30 RPM advance with zero turn command. This bridges the camera forward-mount dead zone.
4. **Ingestion trigger** ($d \leq 200$ mm): Motors pre-spin, chassis advances at fixed 30 RPM. The system transitions to the ingestion confirmation window.

The blind-zone relay condition is formalised as:

$$\neg\texttt{target\_valid} \;\wedge\; (d < 450) \;\wedge\; (d_{\text{last}} < 300) \tag{13}$$

Ingestion Confirmation.

Ingestion is confirmed via a sliding-window majority vote on the distance sensor:

$$\texttt{is\_swallowed} = \left(\sum_{k=1}^{10} \mathbf{1}[d(t + k \cdot 20\,\text{ms}) > 250] \right) \geq 10 \tag{14}$$

i.e., 10 consecutive readings exceeding 250 mm within a 200 ms window, protected by a 3-second hard timeout.

6.4 Navigation & Planning Theory

6.4.1 SLAM (slam_toolbox)

The system employs `slam_toolbox` in Online Asynchronous mode for Simultaneous Localisation and Mapping. The algorithm is built on a sparse pose-graph:

- **Nodes** represent robot poses (x_i, y_i, θ_i) at key scan timestamps.
- **Edges** encode spatial constraints: odometry edges (from wheel odometry) and scan-matching edges (from LiDAR scan alignment).
- **Scan matching** uses Karto-style correlative scan matchers (multi-resolution search over (x, y, θ)) to align incoming LiDAR scans against the accumulated occupancy grid.
- **Global optimisation**: The pose-graph is periodically optimised via non-linear least squares, minimising:

$$\mathbf{X}^* = \arg\min_{\mathbf{X}} \sum_{i,j} \mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij} \tag{15}$$

  where $\mathbf{e}_{ij}$ is the constraint error between nodes i and j and $\mathbf{\Omega}_{ij}$ is the information matrix.
- **Loop closure**: Upon revisiting a previously mapped region, additional scan-matching constraints are added and the full graph is re-optimised.

The resulting map is a 2D occupancy grid, where each cell stores the log-odds $l_{x,y}$ of being occupied:

$$p(\text{occupied} \mid z_{1:t}) = 1 - \frac{1}{1 + \exp(l_{x,y})} \tag{16}$$

6.4.2 AMCL Localisation

Adaptive Monte Carlo Localisation (AMCL) estimates the robot pose $\mathbf{x}_t = (x, y, \theta)_t$ through a particle filter:

1. **Prediction**: Each particle is propagated by sampling from the motion model $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_{t-1})$ using wheel odometry.

2. **Update**: Each particle's weight is updated by the measurement model $p(\mathbf{z}_t \mid \mathbf{x}_t, m)$ using LiDAR scan likelihood against the pre-built map m.

3. **Resampling**: Particles are resampled proportionally to weights (KLD-sampling for adaptive particle count).

6.4.3 Smac Hybrid-A* Global Planner

Nav2 employs the Smac Hybrid-A* planner, which extends classical A* search to a continuous state space. The hybrid state $\mathbf{s} = (x, y, \theta)$ is evaluated with:

$$f(\mathbf{s}) = g(\mathbf{s}) + h(\mathbf{s}) \tag{17}$$

where $g(\mathbf{s})$ is the accumulated cost-to-come (including penalties for proximity to obstacles and non-smooth transitions) and $h(\mathbf{s})$ is an admissible heuristic (typically a 2D Euclidean distance with obstacle-awareness). The continuous-state feasibility check ensures that generated states satisfy the robot's kinematic constraints (minimum turning radius).

6.4.4 Regulated Pure Pursuit Local Controller

The Regulated Pure Pursuit (RPP) controller computes angular velocity ω from a look-ahead point (x_l, y_l) on the global path:

$$\omega = \frac{2v \sin(\alpha)}{L_d} \tag{18}$$

where v is the current linear velocity, α is the angle between the robot's heading and the look-ahead point, and L_d is the adaptive look-ahead distance. The regulation heuristics include velocity scaling near the goal, collision checking on the look-ahead arc, and curvature-based deceleration.

6.4.5 Boustrophedon Coverage Path

The workspace coverage strategy generates an S-shaped (boustrophedon) path through all traversable grid cells. Given a calibrated workspace $\mathcal{W}$ discretised into cells of dimension $\Delta x \times \Delta y$ (matching the intake mechanism width), the path comprises alternating horizontal sweeps:

$$\mathcal{P}_{\text{coverage}} = \left\{ \mathbf{w}_k \in \mathcal{W}_{\text{traversable}} \;\middle|\; k = 1, \ldots, N \right\} \tag{19}$$

Grid cells are classified into one of four states:

Table 17: Grid Cell State Classification

State	Symbol	Semantics
Occupied	■	Static obstacle (unreachable)
Traversable	□	Free space, not yet visited
Cleared	✓	Cell has been swept, no ball remaining
Target	•	Ball detected via YOLO in this cell

6.5 Finite State Machine

The behavioural logic of the CourtSweeper is formally defined as a Mealy-type Finite State Machine (FSM).

Formal Definition.

$$\mathcal{M} = (Q, \Sigma, \delta, q_0, F) \tag{20}$$

where:

- $Q = \{q_0, q_1, q_2, q_3\}$ is the finite set of states.
- Σ is the input alphabet of events (perception events, sensor thresholds, timer expirations, and UI commands).
- $\delta : Q \times \Sigma \to Q$ is the transition function.
- $q_0 =$ `IDLE` is the initial state.
- $F = \emptyset$ (no terminal states; the FSM runs indefinitely).

6.5.1 State Definitions

Table 18: FSM State Set

ID	State	Symbol	Semantics
0	IDLE	q_0	Awaiting command. Manual teleoperation overrides autonomy. Court calibration performed here.
1	SEARCHING	q_1	No ball detected. Chassis follows the precomputed boustrophedon coverage path, marking traversed cells as cleared.
2	TRACKING	q_2	Ball detected. Coverage path paused. Dual-PID controller aligns chassis with target and navigates toward projected map coordinates.
3	INGESTING	q_3	Ball within 200 mm. Roller motors pre-spin; chassis advances; sliding-window detector confirms ingestion.

6.5.2 Event Alphabet

The FSM responds to the following events:

Table 19: FSM Input Events (Σ)

Event	Symbol	Trigger Condition
Ball detected	e_{detect}	`target_locked = True` (YOLO confidence ≥ 0.25)
Ball lost	e_{lost}	`no_ball_counter` ≥ 15 (consecutive frames)
Approach threshold	e_{approach}	$d_{\text{IR}} \leq 200\,\text{mm}$
Ball ingested	e_{ingested}	`is_swallowed = True` (Eq. 14)
Ingestion timeout	e_{timeout}	Elapsed 3 s without ingestion confirmation
Coverage complete	e_{done}	All traversable cells marked cleared
Auto mode	e_{auto}	UI AUTO switch toggled
Manual override	e_{manual}	UI MANUAL switch toggled or teleop input received
E-STOP	e_{estop}	UI E-STOP button pressed

6.5.3 Transition Function

Table 20: FSM Transition Table $\delta : Q \times \Sigma \to Q$

From	Event	To
IDLE	e_{auto}	SEARCHING
IDLE	e_{detect}	TRACKING
SEARCHING	e_{detect}	TRACKING
SEARCHING	e_{done}	IDLE
TRACKING	e_{lost}	SEARCHING
TRACKING	e_{approach}	INGESTING
INGESTING	e_{ingested}	SEARCHING
INGESTING	e_{timeout}	SEARCHING
Any	e_{manual}	IDLE
Any	e_{estop}	IDLE

6.6 Coordinate Transformations

6.6.1 Euler Angle to Quaternion

The robot operates on a planar surface; yaw θ is the only non-zero Euler angle. The conversion to quaternion representation follows the axis-angle formula for a Z-axis rotation:

$$\mathbf{q}(\theta) = (q_x, q_y, q_z, q_w) = \left(0,\ 0,\ \sin\frac{\theta}{2},\ \cos\frac{\theta}{2}\right) \tag{21}$$

6.6.2 Yaw Normalisation

To prevent numerical drift due to θ wrapping outside $[-\pi, \pi]$, the yaw is normalised using:

$$\theta' = \text{atan2}\left(\sin\theta,\ \cos\theta\right) \tag{22}$$

6.6.3 TF Coordinate Tree

The ROS2 TF tree defines the rigid transformations in fig. 21.

Table 21: TF Frame Hierarchy

Parent Frame	Child Frame	Δx	Δy	Δz	$\Delta\theta$	Remarks
`odom`	`base_footprint`	SLAM estimate	SLAM estimate	0	SLAM estimate	AMCL-provided
`base_footprint`	`base_link`	0	0	0	0	Static (coincident origins)
`base_link`	`laser`	0	0	0	0	Static (LiDAR mount)

Coordinate Sign Convention.

- **Mapping mode** (`map_generation.py`): Yaw and x-coordinate are negated ($\theta \rightarrow -\theta$, $x \rightarrow -x$) to align the DJI odometry frame with the SLAM map convention.
- **Navigation mode** (`map_navigation.py`): Raw DJI coordinates are preserved; the saved map already conforms to the localisation frame.

6.7 Communication Protocol Specifications

6.7.1 ROS2 Inter-Process Communication

Table 22: ROS2 Topic Specification

Topic	Message Type	Freq. (Hz)	Direction	Remarks
`/odom`	`nav_msgs/Odometry`	50	Chassis → ROS2	DJI SDK telemetry
`/scan`	`sensor_msgs/LaserScan`	~10	LiDAR → ROS2	RPLidar driver
`/cmd_vel`	`geometry_msgs/Twist`	On demand	ROS2 → Chassis	Nav2 output
`/camera/image/compressed`	`sensor_msgs/CompressedImage`	10	Chassis → ROS2	JPEG, quality 60
`/camera/camera_info`	`sensor_msgs/CameraInfo`	10	Chassis → ROS2	Intrinsic calibration
`/tf`	`tf2_msgs/TFMessage`	50	Bridge → ROS2	Coordinate frames

6.7.2 UDP Control Protocol: iOS to Jetson

Table 23: UDP Command Packet Specification

Command	Payload Format	Semantics
E-STOP	`{"cmd": "estop"}`	Immediate chassis brake, suspend all autonomous logic
RECALL	`{"cmd": "recall"}`	Terminate mission, navigate to origin via Nav2
AUTO	`{"cmd": "auto"}`	Switch FSM to autonomous mode
MANUAL	`{"cmd": "manual"}`	Switch FSM to manual teleoperation
JOYSTICK	`{"vx": f, "vy": f, "vw": f}`	Omnidirectional velocity command

Transport Parameters.

Table 24: UDP Transport Parameters

Parameter	Min.	Typical	Units
Serialisation	—	JSON	—
Packet size (commands)	30	80	bytes
Target latency	5	15	ms

6.7.3 Serial Motor Protocol: Jetson to Arduino

Table 25: Serial Communication Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
Baud rate	—	115200	—	bps	8N1 (8 data, 1 stop, no parity)
Packet format	—	`"L,R\n"`	—	—	CSV-style, newline-terminated
Speed range	-255	—	+255	—	8-bit signed PWM
Encoding	—	UTF-8	—	—	

6.7.4 Wi-Fi Connection Modes

Table 26: Wi-Fi Operation Modes

Mode	SDK Parameter	Use Case
Access Point (AP)	`conn_type="ap"`	Direct iOS connection; Jetson broadcasts SSID for dataset collection and field teleop
Station (STA)	`conn_type="sta"`	Robot/Jetson joins existing Wi-Fi network for SLAM mapping and Nav2 navigation

6.8 Motor Drive Theory

6.8.1 PWM Duty Cycle Control

The Arduino firmware generates Pulse Width Modulation signals with 8-bit resolution. The duty cycle is:

$$D = \frac{|\texttt{speed}|}{255} \times 100\% \tag{23}$$

where $\texttt{speed} \in [-255, 255]$ is the signed command. The effective motor voltage is:

$$V_{\text{motor}} = D \cdot V_{\text{supply}} = D \cdot 11.1\,\text{V} \tag{24}$$

Pre-spin duty ($\texttt{speed} = 90$) corresponds to $D \approx 35\%$, yielding approximately 3.9 V across the 775 motor terminals.

6.8.2 BTS7960 H-Bridge Topology

Each BTS7960 driver module forms a full H-bridge using two half-bridge outputs. The three-wire interface operates as follows:

Table 27: BTS7960 Logic Truth Table

EN	RPWM	LPWM	Motor State	Condition
LOW	0	0	Coast (free-running)	`speed` = 0
HIGH	$\|s\|$	0	Forward rotation	`speed` > 0
HIGH	0	$\|s\|$	Reverse rotation	`speed` < 0

The enable pin (EN) controls the H-bridge output stage: pulling it LOW disconnects both half-bridges, providing a passive coast-stop that prevents back-EMF-induced braking torque.

Pin Mapping.

Table 28: Arduino Pin Assignment for Dual BTS7960 Drivers

Motor	EN	RPWM	LPWM
Left	D4	D5	D6
Right	D7	D9	D10

6.8.3 Communication Watchdog

A hardware-level safety mechanism is implemented in firmware:

$$\text{if } (\texttt{isMoving} \wedge (t_{\text{now}} - t_{\text{lastRecv}} > 500\,\text{ms})) \Rightarrow \text{stopMotor}() \tag{25}$$

If no serial command is received within 500 ms while motors are in motion, all PWM channels are zeroed and the enable pins are pulled LOW. This prevents runaway behaviour in the event of communication loss (e.g., serial cable disconnection, Jetson crash, or software deadlock).

Command Parsing.

The Arduino firmware uses `Serial.parseInt()` to extract signed integers from the CSV stream, validating each packet with a newline terminator check (`Serial.read() == '\n'`). Incomplete or malformed packets are silently discarded, and the last valid speed values are retained.